Theory of Algorithms – Dynamic Programming

Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za





Introduction

- Dynamic Programming is a general algorithm design technique
- Invented to solve optimization problems in 1950s by American mathematician Richard Bellman
- "Programming" ⇔ "planning"



Principle

- Solve problems by storing optimal solutions to overlapping subproblems
  - Each subproblem solved only once and stored for lookup
  - Thus falls into the class of space-time tradeoffs
- Solution derived from a recurrence relation specification
- Must obey principle of optimality
  - An overall optimal solution can be derived from optimal solutions to sub-instances

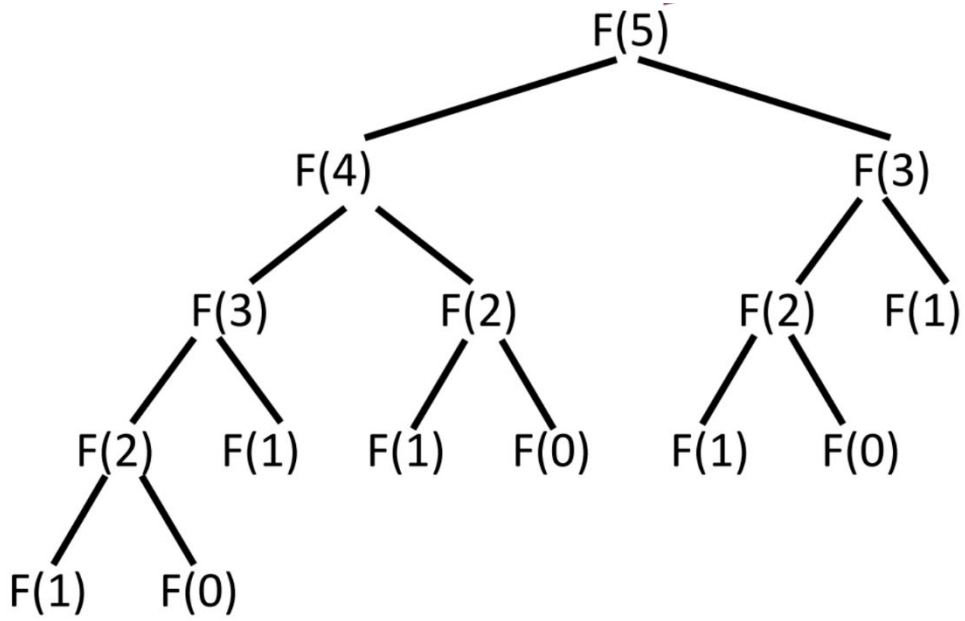
Dynamic Programming Algorithms

- Introductory Algorithms
  - Fibonacci
  - Change-making
  - Coin collecting problem
- Transitive Closure: Warshall's Algorithm
- All Pairs Shortest Path: Floyd's Algorithm
- Knapsack Problem

Fibonacci

- F(n) = F(n-1)+F(n-2)
- Brute Force

```
fibo(n):
      if (n == 0) return 0
      if (n == 1) return 1
      return fibo(n-1) + fibo(n-2)
```



- Dynamic Programming

```
fibo(n):
      F[0] = 0
      F[1] = 1
      for i = 2 to n do:
            F[i] = F[i - 1] + F[i - 2]
      return F[n]
```

F[0]	F[1]	F[2]	F[3]	F[4]	F[5]
0	1	1	2	3	5

Coin-row Problem I

- The problem:
  - given a row of coins with face values $c_1 \ldots c_n$
  - We want to pick up coins that result in **largest value** BUT we are not allowed to pick up coins next to each other (in original row – so we first make a selection, then pick them up simultaneously)
- Define F(n) as the max amount we can up from these n coins
- Recurrence Relation
  - Approach: for n coins, split allowed coin selection into two parts, then take max of these options
    - solutions that **included the last coin**: F(n-2) + c_n
    - solutions without the last coin: F(n-1)
  - $F(0) = 0, F(1) = c_1$
  - $F(n) = \max \{F(n-2) + c_n,\ F(n-1)\} \ \ for\ n > 1$

$F[0] \leftarrow 0; \quad F[1] \leftarrow C[1]$
for $i \leftarrow 2$ **to** n **do**
    $F[i] \leftarrow \max(C[i] + F[i-2],\ F[i-1])$
return $F[n]$

Coin-row Problem II

- Here the array C shows the coin value at each position (this is just to help visualize soln)
- Just fill array F from left to right
- At each step evaluate

  max{c(n) + F(n-2), F(n-1)}
- Backtrack manually:

1. Start at 17 – what choice led to that ? You can see we made choice that ADDS a coin – c(6) here, this requires us to have evaluated F(n-2) = F(4) (n=6)
2. So, we go to that cell. Then ask what choice led to this value? Again, we chose the option that adds a coin – c(4) here – and this required F(n-2) = F(2) to be computed (n=4)
3. So we go to F(2), but this used previous solution, F(n-1) = F(1) (n=2) .
4. So we look there and this gives us the last coin (we terminate here) – that coin is c(1)

- So solution set is {c1, c4, c6}
- For later backtracking, to get solution, can store choice at each step

$F[0] = 0,\ F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5					

$F[2] = \max\{1 + 0, 5\} = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5				

$F[3] = \max\{2 + 5, 5\} = 7$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7			

$F[4] = \max\{10 + 5, 7\} = 15$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15		

$F[5] = \max\{6 + 7, 15\} = 15$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	

$F[6] = \max\{2 + 15, 15\} = 17$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	**17**

Change Making Problem I

- Given a collection of coin denominations $d_1 < d_2 < \ldots < d_m$
  - Find exact change for the amount *n* such that the minimum number of coins is used
  - Each denomination has an unlimited number
  - $d_1 = 1$
- For typical sub-cases there is an efficient greedy algorithm
  - Here we consider the general case



Change Making Problem II

- Recurrence Relation
  - Add one coin that accounts for the largest part of n for m different denominations, d_j
  - *F(n)* is minimum number of coins adding up to *n*
  - $F(0) = 0$
  - $F(n) = \min_{j:n \geq d_j}\{F(n - d_j)\} + 1 \ \ for\ n > 0$
- Algorithm:
  - Iterate from 1 to n calculating F

```
F[0] ← 0
for i ← 1 to n do
    temp←∞; j←1
    while j ≤ m and i ≥ D[j] do
        temp ← min(F[i − D[j]], temp)
        j←j+1
    F[i] ← temp + 1
return F [n]
```

$$F(n) = \min_{j:n \geq d_j} \{F(n - d_j)\} + 1$$

Change Making Problem III

- n=6, D = [1,3,4], F[0] = 0
  - i=1: F[1]=min{F[1-1]}+1=1
  - i=2: F[2]=min{F[2-1]}+1=2
  - i=3: F[3]=min{F[3-1],F[3-3]}+1=1
  - i=4: F[4]=min{F[4-1],F[4-3],F[4-4]}+1=1
  - i=5: F[5]=min{F[5-1],F[5-3],F[5-4]}+1=2
  - i=6: F[6]=min{F[6-1],F[6-3],F[6-4]}+1=2

Then backtrack to find coins – 2 x 3-coins here (=6 value) – see F[i] above

0	1	2	3	4	5	6	*n*
0	1						*F*

0	1	2	3	4	5	6	*n*
0	1	2					*F*

0	1	2	3	4	5	6	*n*
0	1	2	1				*F*

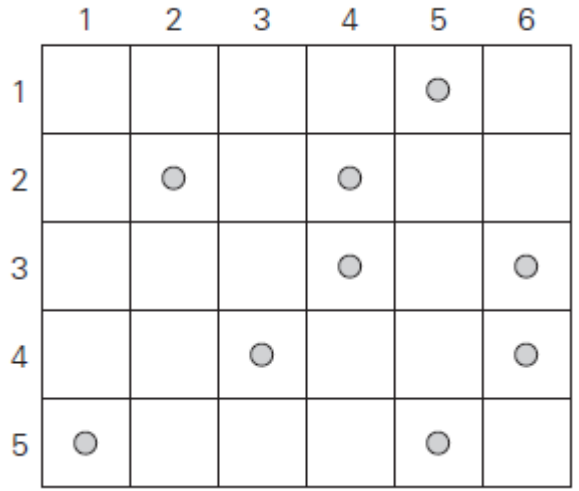
0	1	2	3	4	5	6	*n*
0	1	2	1	1			*F*

0	1	2	3	4	5	6	*n*
0	1	2	1	1	2		*F*

0	1	2	3	4	5	6	*n*
0	1	2	1	1	2	2	*F*

Coin collecting problem I

- Given a board with n x m cells, some cells have 1 coin placed (0 or 1 allowed)
- A robot must start at top-left and collect coins as it moves to bottom right cell
  - BUT robot can only move one cell to right, **or** one cell down as it moves. Must pick up a coin in a cell if it visits
- Problem: Find maximum number of coins it can collect and the path that it follows



Coin collecting problem II

Formalize problem:

- Let F(i, j) be the largest number of coins the robot can collect and bring to cell (i, j) in the ith row and j^{th} column of the board
  - Can reach cell (i,j)from ONLY (i-1,j) or (i, j-1) (i.e. left or above)
  - Largest # coins carried to those cells already are F(i-1,j) and F(i,j-1)
  - So, largest number of coins at (i, j) is the max of coins carried from a route going through either of those cells PLUS any coin added at cell (i,j)
  - We also need initial conditions: F(i,0) = 0, F(0,j) = 0 from 0th row and column (not shown in table – used to initialize row 1 and column 1)
- So, the final recurrence relation is:

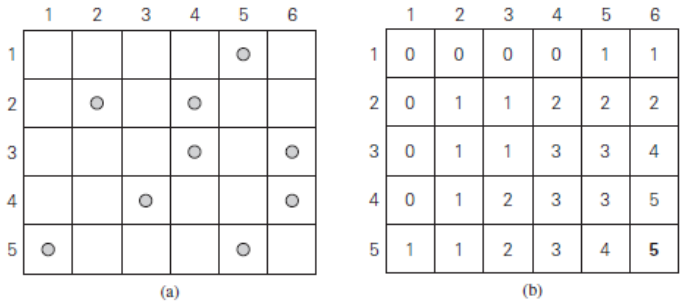
$$F(i,j) = \max\{ F(i-1,j) , F(i,j-1) \} + c_{ij}, \quad 1 \le i \le n, \ 1 \le j \le m$$

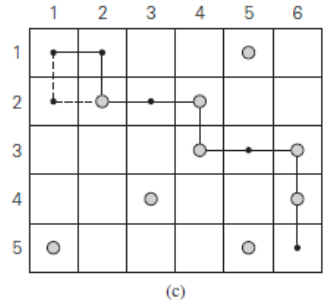
where c_{ij} = 1or 0, if coin present or not;

Coin collecting problem III

- Fill board row by row by evaluating recurrence
- Time and space efficiency Θ(nm)
- Backtrace for path(s) – start at end goal state F(5,6) and see how we got there
  - If F(i-1,j) > F(i,j-1), we came from above
  - If F(i,j-1) > F(i-1,j), we came from, left
  - If F's the same we choose either
- Backtrace: Θ(n+m)

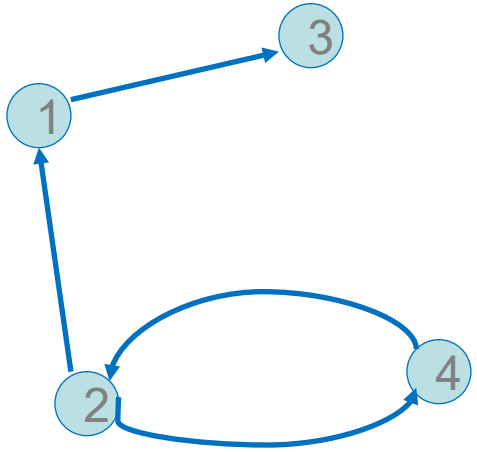
RobotCoinCollection(C[1..*n*, 1..*m*])
//In: Matrix *C*[1..*n*, 1..*m*] - coin mx (0 or 1)
//Out: Largest # coins the robot can bring to cell *(n, m)*
F[1, 1]←*C*[1, 1];
for *j* ←2 **to** *m* **do** *F*[1, *j*]←*F*[1, *j* − 1]+ *C*[1, *j*]
for *i* ←2 **to** *n* **do**
    F[*i*, 1]←*F*[*i* − 1, 1]+ *C*[*i*, 1]
    for *j* ←2 **to** *m* **do**
        F[*i*, *j*]←max(*F* [*i* − 1, *j*], *F*[*i*, *j* − 1]) + *C*[*i*, *j*]
return *F*[*n*, *m*]





Transitive Closure

- For a directed graph with *n* vertices
- Transitive closure is the *n*-by-*n* Boolean matrix $T = \{t_{ij}\}$ indicating reachability
  - If there is a 1 in the ith row and j^{th} column then there is a directed path from vertex *i* to vertex *j*
  - If the entry is 0 then there is no path
- Example:



$$T = \begin{array}{c} \\ \text{from} \end{array} \begin{array}{c} \text{to} \\ \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{matrix} \end{array}$$

Warshall's Algorithm Principle

- Idea: if there's a way to go from A to B, and also from B to C, then there's a way to get from A to C
- A path exists between two vertices i, j, *iff* there is
  - an edge from i to j; or
  - a path from i to j going through vertex 1; or
  - a path from i to j going through vertex 1 and/or 2; or
  - a path from i to j going through vertex 1, 2, and/or 3; or
  - ...
  - a path from i to j going through any of the other vertices
- Successively build corresponding transition matrices $R^{(0)}$, $R^{(1)}$, …, $R^{(n)} = T$

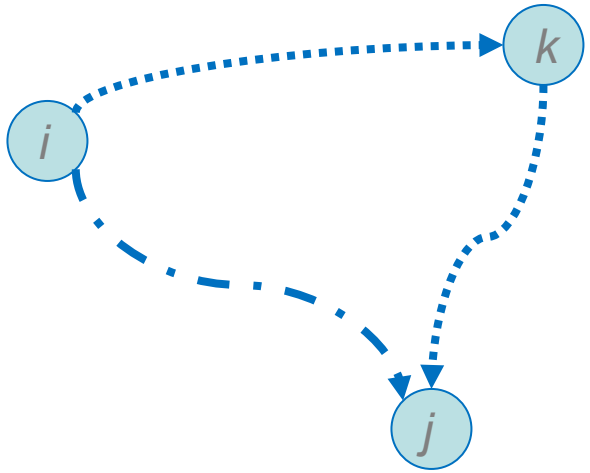
Warshall's Algorithm

- In the kth stage determine if a path exists between two vertices i, j using just vertices among 1,…,k

$$R^{(k)}[i,j] = \begin{cases} R^{(k-1)}[i,j] \\ \textbf{or} \\ R^{(k-1)}[i,k] \textbf{ and } R^{(k-1)}[k,j] \end{cases}$$

(path only via $1, \ldots, k-1$)

$$\begin{pmatrix} \text{path from i to k} \\ \text{and from k to j} \\ \text{using just } 1, \ldots, k-1 \end{pmatrix}$$



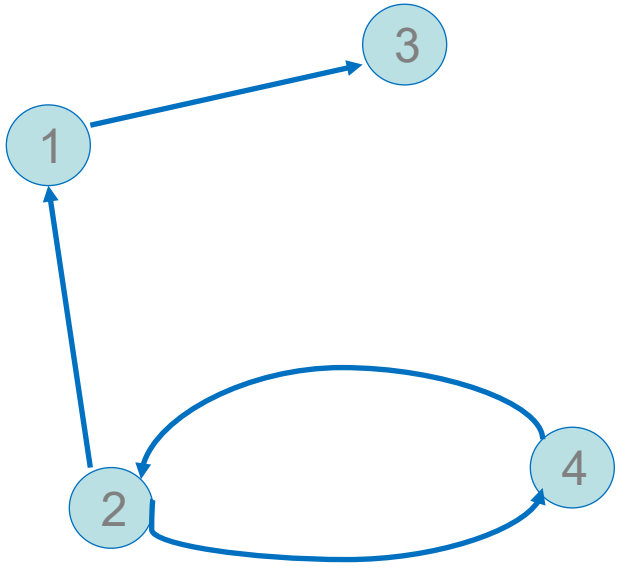
k^{th} stage

Warshall's Algorithm Code

```
R(0) ← adjacency matrix
for k ← 1 to n do
    for  i ← 1 to n do
        for  j ← 1 to n do
            R(k)[i,j] ← R(k-1)[i,j] or (R(k-1)[i,k] and R(k-1)[k,j])
return R(n)
```

- Complexity:
  - Time efficiency in $\Theta(n^3)$
  - Space efficiency in $\Theta(n^3)$ – can reduce to $\Theta(n^2)$

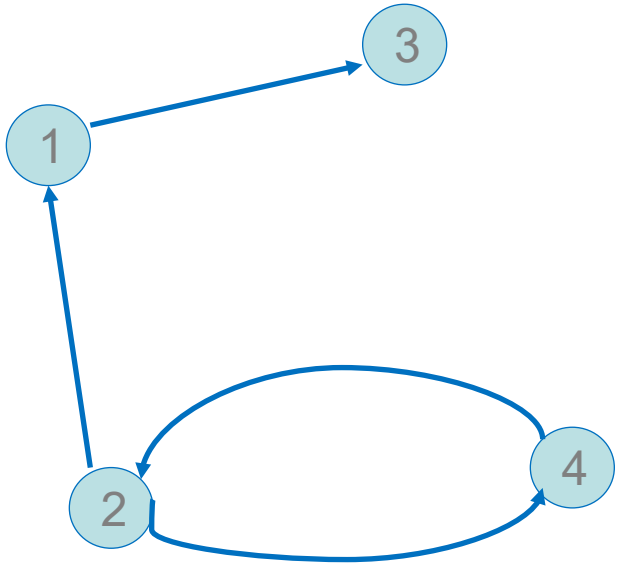
Warshall's Algorithm Example: R^0



$$R^0 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{matrix}$$

- Note R^0 is the Adjacency Matrix
  - The *n*-by-*n* Boolean matrix where a 1 in row i column j indicates a directed edge from i to j

Warshall's Algorithm Example: R^1

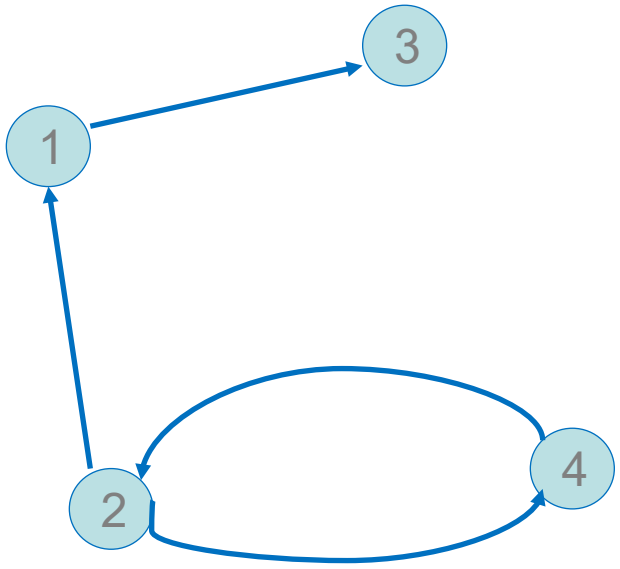


$$R^0 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{matrix}$$

$$R^1 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{matrix}$$

- All paths from i to j, when using node index 1 is permitted
- Change Boolean from 0 to 1 at (i, j)where column j in row 1 and row i in column 1 both have 1's

Warshall's Algorithm Example: R^2

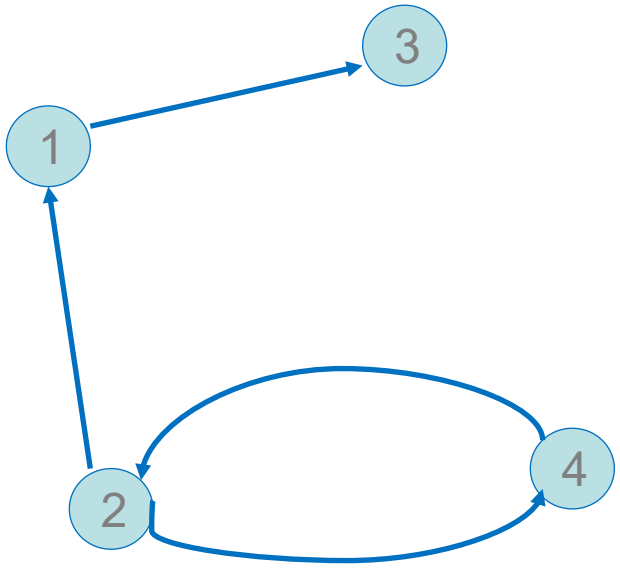


$$R^1 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{matrix}$$

$$R^2 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{matrix}$$

- All paths from i to j passing through node 1 and/or 2
- Change Boolean from 0 to 1 at (i, j)where column j in row 2 and row i in column 2 both have 1's

Warshall's Algorithm Example: R^3

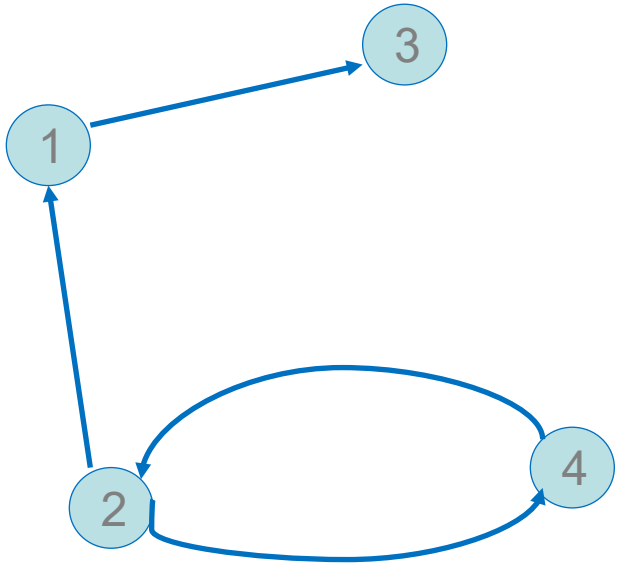


$$R^2 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{matrix}$$

$$R^3 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{matrix}$$

- All paths from i to j passing through nodes 1, 2, 3 (potentially)
- Change Boolean from 0 to 1 at (i, j)where column j in row 3 and row i in column 3 both have 1's
  - No new paths because vertex 3 is a sink

Warshall's Algorithm Example: R^4



$$R^3 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{matrix}$$

$$T = R^4 = \begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{matrix}$$

- All paths from i to j passing through nodes 1 to 4 (potentially)
- Change Boolean from 0 to 1 at (i, j)where column j in row 4 and row i in column 4 both have 1's